UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

CSE 402: Operating System Lab Report 03

Preemptive Shortest Job First (SRTF) Scheduling Simulation

Student Name: Faiaz Masnun Labib
Student ID: 23101128
Section: C-1

Instructor: Atia Rahman Orthi
Designation: Lecturer, Dept. of CSE
Date of Submission: September 04, 2026

1. Problem Statement

Analyze the scheduling execution of four independent processes using the **Preemptive Shortest Job First (SRTF)** paradigm based on the specifications below:

Process ID	Arrival Time (AT)	Burst Time (BT)
P1	0	4
P2	1	2
P3	3	2
P4	2	1

Required Deliverables:

1. Establish the discrete Gantt chart execution chronology.
2. Compute individual Completion Time (CT), Turnaround Time (TAT = CT − AT), and Waiting Time (WT = TAT − BT).
3. Determine system-wide Average Turnaround Time and Average Waiting Time.

2. Python Implementation

```
pro = [
    ["P1", 0, 4],
    ["P2", 1, 2],
    ["P3", 3, 2],
    ["P4", 2, 1]
]

remaining_bt = {p[0]: p[2] for p in pro}
process_dict = {p[0]: {"at": p[1], "bt": p[2]} for p in pro}

time = 0
completed = 0
n = len(pro)
execution_order = []
ct_dict = {}

while completed < n:
    available = [
        pid for pid, info in process_dict.items()
        if info["at"] <= time and remaining_bt[pid] > 0
    ]

    if not available:
        time += 1
```

```
        continue

    chosen_pid = min(available, key=lambda x: (remaining_bt[x],
        process_dict[x]["at"]))

    remaining_bt[chosen_pid] -= 1

    if not execution_order or execution_order[-1] != chosen_pid:
        execution_order.append(chosen_pid)

    time += 1

    if remaining_bt[chosen_pid] == 0:
        ct_dict[chosen_pid] = time
        completed += 1

total_tat, total_wt = 0, 0
print(f"{'PID':<6}{'AT':<6}{'BT':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
for p in pro:
    pid, at, bt = p
    ct = ct_dict[pid]
    tat = ct - at
    wt = tat - bt
    total_tat += tat
    total_wt += wt
    print(f"{pid:<6}{at:<6}{bt:<6}{ct:<6}{tat:<6}{wt:<6}")

print(f"\nAverage TAT = {total_tat / n:.2f}")
print(f"Average WT = {total_wt / n:.2f}")

print("\nExecution Order (Gantt Chart):")
for pid in execution_order:
    print(pid, end=" --> ")
print("End")
```

GitHub Repository Link: Click Here

3. Simulation Output

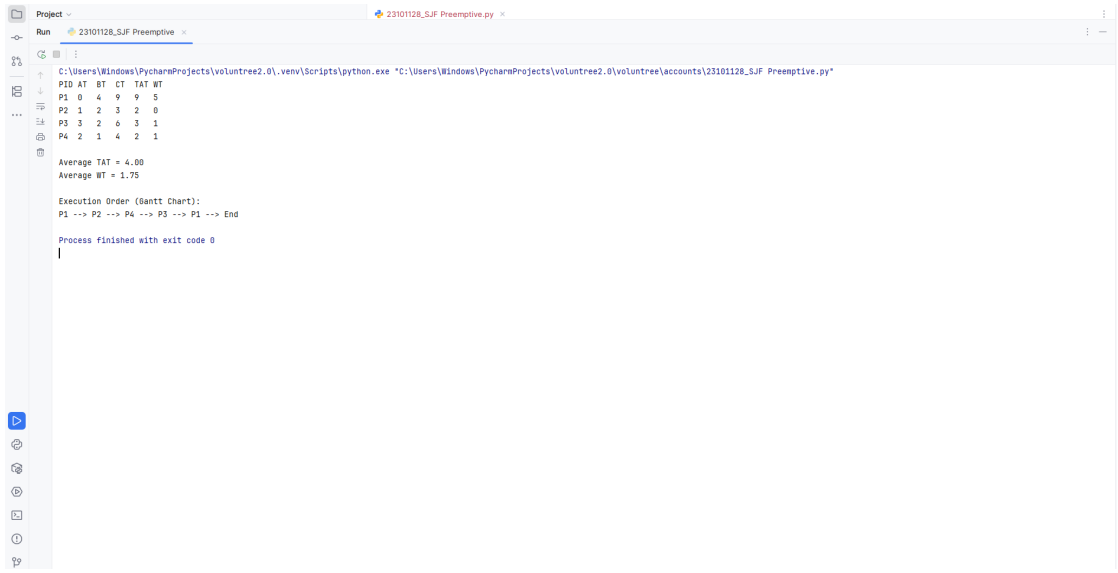

Figure 1: PyCharm Execution Output for SRTF Scheduling

4. Results and Performance Analysis

4.1 Process Metric Summary

Process ID	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	4	9	9	5
P2	1	2	3	2	0
P3	3	2	6	3	1
P4	2	1	4	2	1
Mean Turnaround Time = 4.00 Units			**Mean Waiting Time = 1.75 Units**		

4.2 Chronological Timeline Trace

Time-Slice Transition Matrix

- **Interval** $[0-1]$**:** Initial state. P_1 has sole ownership of the CPU, reducing its remaining demand from $4 \rightarrow 3$ units.
- **Interval** $[1-3]$**:** P_2 enters at $t = 1$ requiring 2 units. Since $2 < 3$, it triggers preemption on P_1. At $t = 2$, P_4 enters (BT $= 1$) creating a tie with the remaining 1 unit of P_2. The scheduler favors P_2 via arrival order (AT $= 1$ vs 2), letting P_2 conclude at $t = 3$.
- **Interval** $[3-4]$**:** At $t = 3$, P_3 arrives (BT $= 2$). P_4 has the lowest remaining demand (1) among ready processes ($P_4 : 1, P_3 : 2, P_1 : 3$) and completes at $t = 4$.
- **Interval** $[4-6]$**:** Between remaining candidates P_3 (2) and P_1 (3), P_3 claims the processor and completes at $t = 6$.
- **Interval** $[6-9]$**:** With all shorter jobs flushed, the suspended P_1 resumes to clear its outstanding 3 units, finalizing at $t = 9$.

5. Discussion and Analytical Observations

- **Suppression of Convoy Delay:** Had non-preemptive logic governed the queue, P_1 would have retained execution across $[0-4]$, causing short incoming tasks like P_2 and P_4 to accumulate unnecessary latency. SRTF immediate preemption mitigates this head-of-line penalty.
- **System Responsiveness:** Allowing jobs with lighter computational demands to leapfrog longer tasks yields an average turnaround of 4.00 units and average wait of only 1.75 units.
- **Operational Overhead:** Preemptive efficiency incurs a runtime penalty: context switching overhead arises each time an active thread is swapped out (seen here when suspending P_1 at $t = 1$ and restoring it at $t = 6$).